

ĐỀ BÀI THỰC HÀNH

Cài đặt Gaussian Naive Bayes từ đầu bằng NumPy trên bộ dữ liệu Iris

1 Mục tiêu

- Hiểu rõ nguyên lý hoạt động của thuật toán **Naive Bayes**.
- Biết cách áp dụng Naive Bayes với giả định phân phối **Gaussian (chuẩn)** cho dữ liệu liên tục.
- Luyện tập cài đặt một thuật toán học máy từ đầu **chỉ với NumPy**, không dùng thư viện học máy có sẵn như `scikit-learn`.

2 Kiến thức cần nắm

- Định lý Bayes:

$$P(y|x) \propto P(y) \cdot P(x|y)$$

- Giả định **độc lập có điều kiện** giữa các đặc trưng trong Naive Bayes.
- Hàm mật độ Gaussian:

$$P(x_i|y) = \frac{1}{\sqrt{2\pi\sigma_{y,i}^2}} \exp\left(-\frac{(x_i - \mu_{y,i})^2}{2\sigma_{y,i}^2}\right)$$

3 Bộ dữ liệu sử dụng

- **Iris dataset** (có sẵn trong thư viện `sklearn.datasets`):
 - Số mẫu: 150
 - Số đặc trưng: 4 (chiều dài và chiều rộng đài hoa, cánh hoa)
 - Số lớp: 3 (Setosa, Versicolor, Virginica)

4 Yêu cầu bài tập

Phần 1: Chuẩn bị dữ liệu

1. Load dữ liệu Iris bằng `sklearn.datasets.load_iris()`.
2. Tách dữ liệu thành tập huấn luyện (80%) và tập kiểm tra (20%) bằng `train_test_split`.

Phần 2: Cài đặt Gaussian Naive Bayes

Hoàn thiện lớp `GaussianNaiveBayes` với các phương thức sau (sử dụng **NumPy**):

(1) `fit(X, y)`

- Input:
 - `X`: mảng (`n_samples, n_features`)
 - `y`: mảng (`n_samples,`)
- Công việc:
 - Tìm danh sách các lớp.
 - Với mỗi lớp:
 - * Tính vector **trung bình (mean)** (`n_features,`).
 - * Tính vector **phương sai (var)** (`n_features,`).
 - * Tính **prior** = tỷ lệ số mẫu lớp đó / tổng số mẫu.
- Lưu kết quả vào `self.params` và `self.priors`.

(2) `gaussian_pdf(x, mean, var)`

- Input:
 - `x`: vector một mẫu (`n_features,`)
 - `mean, var`: vector trung bình và phương sai của lớp (`n_features,`)
- Output:
 - `pdf`: vector (`n_features,`), mỗi phần tử là xác suất của một đặc trưng theo Gaussian.

(3) `predict_single(x)`

- Input: `x` một mẫu (`n_features,`)
- Công việc:
 - Với mỗi lớp `c`:
 - * Tính $\log(P(y=c))$
 - * Tính $\log(P(x|y=c)) = \text{tổng log của các xác suất đặc trưng}$

- * Cộng lại thành posterior
- Trả về lớp có posterior lớn nhất.

- Output: nhãn dự đoán (số nguyên).

(4) `predict(X)`

- Input: `X` nhiều mẫu (`n_samples`, `n_features`)
- Output: vector nhãn dự đoán (`n_samples`,)
- Gợi ý: gọi `predict_single` cho từng hàng của `X`.

Phần 3: Đánh giá

1. Dự đoán nhãn cho tập test.
2. Tính độ chính xác (`accuracy`).
3. In ra nhãn dự đoán so với nhãn thật.

5 Yêu cầu nộp bài

- File code Python `.py` hoặc Notebook `.ipynb`.
- Sinh viên cần hoàn thiện **phần còn thiếu trong khung code** (các hàm `fit`, `gaussian_pdf`, `predict_single`, `predict`).
- Chạy thử với dữ liệu Iris và chụp lại kết quả (`accuracy` và so sánh nhãn).

6 Gợi ý kiểm tra kết quả

- Độ chính xác mong đợi trên Iris thường đạt **>90%** (nếu cài đúng).
- Có thể so sánh với `sklearn.naive_bayes.GaussianNB` để kiểm chứng.